Compare & Contrast

Dual Implementation of an Expense Approval Workflow

Azure Durable Functions vs. Azure Logic Apps + Service Bus

CST8917 – Serverless Applications | Assignment 2

1. Workflow & Business Rules

The same business process is implemented twice

Input
Employee name/email, amount, category, description, manager email

Validation
Required fields + six valid categories

Under $100
Automatically approved; no manager interaction

$100 or more
Wait for manager Approve / Reject

Timeout
Auto-approve and mark Escalated = Yes

Notification
Employee receives the final outcome by email

2. Version A — Azure Durable Functions

Code-first orchestration using the Python v2 programming model

HTTP client starts a new expense orchestration.

Orchestrator coordinates validation, processing and notification activities.

Expenses ≥ $100 use the Human Interaction pattern.

External event represents the manager's Approve / Reject decision.

Durable timer races against the external event for timeout escalation.

Durable state allows the orchestration to wait without keeping a worker continuously active.

Key design decision

Use wait_for_external_event + durable timer to model manager interaction naturally.

3. Version A — Demo Plan

Show the Human Interaction pattern, not every test case live

Live Demo A1

Submit an expense ≥ $100 →
show orchestration waiting →
call manager approval endpoint →
show completion.

Live Demo A2

Show timeout/escalation evidence:
no manager response →
timer wins →
auto-approved with Escalated = Yes.

Evidence

Use test-durable.http / screenshots
to show all six scenarios were covered.

Point out the orchestrator, activity functions, client function and manager-response endpoint.

Explain why the durable timer is important: the workflow can wait reliably for a long-running human decision.

Show final status/output or notification so the audience sees the end-to-end result.

4. Version B — Logic Apps + Service Bus

Visual/declarative orchestration with event-driven messaging

Service Bus queue receives incoming expense requests.

Logic App trigger consumes the queue message.

Azure Function validates required fields and category.

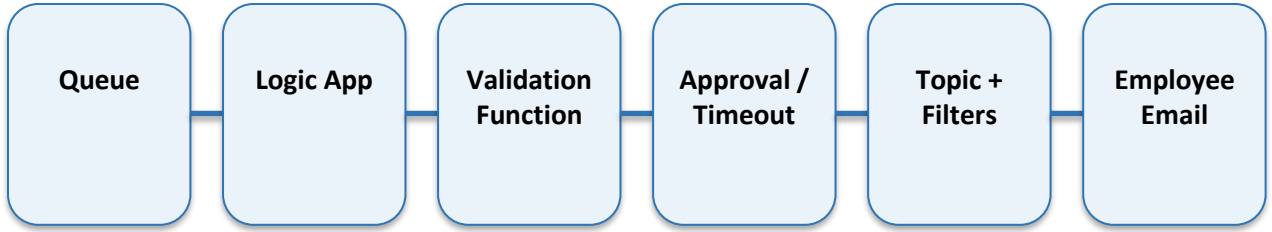
Condition branches handle validation and the $100 threshold.

Manager approval uses 'Send email with options' for Approve / Reject.

Timeout path auto-approves and marks the request escalated.

Outcome messages are routed through a Service Bus topic with approved, rejected and escalated subscriptions.

Employee receives the final outcome by email.



What I experienced

The visual run history made branch execution easy to see, but data-shape problems (Base64/JSON/null values) required careful debugging.

5. Version B — Demo Evidence

Use the Azure portal run history plus the received emails

Scenario 1
$50 valid expense →
validation succeeds →
auto-approved →
employee receives approval email.

Manager decision
$450 expense →
approval email →
manager selects Approve or Reject →
correct condition branch executes.

Scenarios 5 & 6
Missing description / invalid category →
validation fails →
employee receives a Validation error email.

Show a successful Logic App run with green check marks through the executed branch.

Show the approval/rejection email and the final employee notification.

Show topic subscription counts or Service Bus evidence for approved, rejected and escalated outcomes.

Briefly mention that early failures helped reveal JSON decoding and null-field issues in the visual workflow.

6. Comparison — Six Required Dimensions

Direct comparison based on implementation experience

Dimension	Durable Functions	Logic Apps + Service Bus
Development	More code to write; precise control	Fast visual assembly; connector setup adds complexity
Testability	Strong local testing with .http and code	More dependent on deployed connectors/services
Error handling	Fine-grained retries/exceptions in code	Built-in action status/retry configuration; visual failures
Human interaction	Natural: external event + durable timer	Possible with email options + timeout, but less native
Observability	Logs/status endpoints; code-oriented	Excellent visual run history and per-action I/O
Cost	Consumption-based Functions + storage	Per-action/connector costs + Service Bus + validation Function

7. Cost — Compare the Scaling Model

Final dollar estimates should use the Azure Pricing Calculator and stated assumptions

~100 expenses/day

Low volume. Both consumption-oriented approaches are inexpensive; free grants/minimum service charges and connector usage can dominate the estimate.

~10,000 expenses/day

Action count, Service Bus operations, Function executions, storage transactions, email/connector calls and approval duration become important.

State assumptions: average actions per expense, percentage requiring manager approval, timeout percentage, Function execution duration/memory, and Service Bus tier.

Durable Functions: include Functions consumption plus Durable storage transactions.

Logic Apps: include trigger/action executions, Service Bus operations, validation Function, and any connector-related charges.

Do not claim one is always cheaper—the workflow shape and action count determine the result.

8. Recommendation

Choose based on workflow complexity, team skills and operational needs

Primary recommendation: Durable Functions

For a production workflow with complex state,
custom business rules, automated testing and
long-running human interaction,
I would favor Durable Functions
because the external-event + durable-timer model is explicit
and code is easier to version and test.

When I would choose Logic Apps

For integration-heavy workflows where visibility,
rapid visual development and managed connectors
are more important than code-level control,
Logic Apps is attractive—especially for teams that need operations
staff to understand the workflow visually.

The decision is not 'code good, visual bad.' Each approach optimizes a different development and operations experience.

My implementation showed that the Human Interaction pattern was more natural in Durable Functions, while troubleshooting branches was more immediately visible in Logic Apps.

9. Lessons Learned

What changed after implementing the same workflow twice

Implementing the same business rules twice made the orchestration trade-offs much clearer than studying the services separately.

Durable Functions hides much of the complexity of waiting: the orchestration can pause for a human event and resume later.

Logic Apps made execution paths highly visible, but message encoding, JSON conversion and null values can break downstream actions if the data contract is not consistent.

Service Bus decouples producers from the workflow and makes outcome routing extensible through filtered topic subscriptions.

Next time, I would define and test the expense JSON schema first, automate more tests, and standardize action names before building the full workflow.

The best architecture depends on maintainability, testing, integration needs, observability and cost—not only which implementation is faster to build.